# SQL JOIN and GROUP BY Lab - Student Exercises

## Lab Introduction

This lab is designed to help you practice SQL JOINs and GROUP BY clauses through hands-on exercises. You will work with a company database containing information about departments, employees, customers, orders, products, and order details.

## Database Schema Overview

The database contains the following tables:

**departments**

- dept_id (Primary Key)
- dept_name
- location

**employees**

- employee_id (Primary Key)
- first_name
- last_name
- email
- hire_date
- salary
- dept_id (Foreign Key → departments)
- manager_id (Foreign Key → employees)

**customers**

- customer_id (Primary Key)
- customer_name
- email
- city
- country

**orders**

- order_id (Primary Key)
- customer_id (Foreign Key → customers)
- order_date
- total_amount
- status

**products**

- product_id (Primary Key)

- product_name

- category

- unit_price

**order_details**

- order_detail_id (Primary Key)

- order_id (Foreign Key → orders)

- product_id (Foreign Key → products)

- quantity

- unit_price

## Setup Instructions

Before starting the exercises, run the setup script provided by your trainer to create the database tables and insert sample data.

## Exercises

### Exercise 1: Basic INNER JOIN

List all employees with their department names. Display employee ID, first name, last name, and department name. Order the results by employee ID.

**Hint:** You need to join the employees and departments tables.

**Write your query here**

```
SELECT
    e.employee_id,
    e.first_name,
    e.last_name,
    d.dept_name
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.dept_id
ORDER BY e.employee_id;
```

**Expected columns:** employee_id, first_name, last_name, dept_name

**Output:**

Result Grid | Filter Rows: | Expo

| employee_id | first_name | last_name | dept_name |
|---|---|---|---|
| 101 | John | Smith | IT |
| 102 | Mary | Jones | Finance |
| 103 | David | Brown | IT |
| 104 | Linda | Taylor | HR |
| 105 | James | Wilson | Sales |
| 106 | Susan | Thomas | Sales |
| 107 | Robert | Martin | Marketing |
| 108 | Karen | Lee | Finance |
| 109 | Michael | Clark | IT |
| 110 | Nancy | Hall | HR |
| 111 | Daniel | Young | Marketing |
| 112 | Emma | King | Finance |

---

**### Exercise 2: LEFT JOIN to Find Missing Data**

Find all customers and their order count, including customers who haven't placed any orders. Display customer ID, customer name, and the count of orders. Order by order count in descending order.

****Hint:**** Use LEFT JOIN to include customers with no orders, and COUNT with GROUP BY.

**-- Write your query here**

```
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY order_count DESC;
```

****Expected columns:**** customer_id, customer_name, order_count

---

**Output:**

Result Grid | Filter Rows:

| customer_id | customer_name | order_count |
|---|---|---|
| 201 | Alice Green | 2 |
| 202 | Bob White | 2 |
| 203 | Charlie Black | 2 |
| 204 | Diana Blue | 2 |
| 205 | Ethan Gray | 1 |
| 206 | Fiona Brown | 1 |

**### Exercise 3: Multiple JOINs**

List all orders with customer names and the total number of items in each order. Display order ID, customer name, order date, total amount, count of different items, and sum of quantities. Order by order ID.

****Hint:**** You need to join three tables: orders, customers, and order_details.

**-- Write your query here**

```
SELECT
    o.order_id,
    c.customer_name,
    o.order_date,
    o.total_amount,
    COUNT(od.product_id) AS item_count,
    SUM(od.quantity) AS total_quantity
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
JOIN order_details od
    ON o.order_id = od.order_id
GROUP BY
    o.order_id,
    c.customer_name,
    o.order_date,
    o.total_amount
ORDER BY o.order_id;
```

**Expected columns:** order_id, customer_name, order_date, total_amount, item_count, total_quantity

**Output:**

| order_id | customer_name | order_date | total_amount | item_count | total_quantity |
|---|---|---|---|---|---|
| 401 | Alice Green | 2024-01-10 | 925.00 | 2 | 2 |
| 402 | Bob White | 2024-01-15 | 70.00 | 1 | 1 |
| 403 | Charlie Black | 2024-02-05 | 245.00 | 2 | 2 |
| 404 | Diana Blue | 2024-02-18 | 120.00 | 1 | 1 |
| 405 | Ethan Gray | 2024-03-01 | 900.00 | 1 | 1 |
| 406 | Fiona Brown | 2024-03-10 | 200.00 | 1 | 1 |
| 407 | Alice Green | 2024-03-15 | 95.00 | 1 | 2 |
| 408 | Bob White | 2024-04-01 | 70.00 | 1 | 1 |
| 409 | Charlie Black | 2024-04-05 | 120.00 | 1 | 1 |
| 410 | Diana Blue | 2024-04-12 | 200.00 | 3 | 3 |

### Exercise 4: GROUP BY with Aggregates

Calculate the average salary by department. Display department name, count of employees, average salary, minimum salary, and maximum salary. Order by average salary in descending order.

**Hint:** Join departments and employees, then use GROUP BY with aggregate functions.

```sql
-- Write your query here
SELECT
    d.dept_name,
    COUNT(e.employee_id) AS employee_count,
    AVG(e.salary) AS avg_salary,
    MIN(e.salary) AS min_salary,
    MAX(e.salary) AS max_salary
FROM departments d
JOIN employees e
    ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY avg_salary DESC;
```

**Expected columns:** dept_name, employee_count, avg_salary, min_salary, max_salary

**Output:**

| | dept_name | employee_count | avg_salary | min_salary | max_salary |
|---|---|---|---|---|---|
| ▶ | HR | 2 | 82500.000000 | 80000.00 | 85000.00 |
| | IT | 3 | 70666.666667 | 65000.00 | 75000.00 |
| | Finance | 3 | 66666.666667 | 62000.00 | 70000.00 |
| | Sales | 2 | 57500.000000 | 55000.00 | 60000.00 |
| | Marketing | 2 | 51000.000000 | 50000.00 | 52000.00 |

**### Exercise 5: HAVING Clause**

Find departments with an average salary greater than 65000 and more than 2 employees. Display department name, employee count, and average salary. Order by average salary in descending order.

****Hint:**** Use HAVING clause to filter groups after aggregation.

**```sql**

**-- Write your query here**

```
SELECT
   d.dept_name,
   COUNT(e.employee_id) AS employee_count,
   AVG(e.salary) AS avg_salary
FROM departments d
JOIN employees e
   ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > 65000
  AND COUNT(e.employee_id) > 2
ORDER BY avg_salary DESC;
```

****Expected columns:**** dept_name, employee_count, avg_salary

**Output:**

| | dept_name | employee_count | avg_salary |
|---|---|---|---|
| ▶ | IT | 3 | 70666.666667 |
| | Finance | 3 | 66666.666667 |

### Exercise 6: SELF JOIN

List all employees with their manager's name. Display employee ID, employee full name, employee salary, manager full name, and manager salary. Order by employee ID.

**Hint:** Join the employees table to itself. Use LEFT JOIN to include employees without managers.

```sql
-- Write your query here
SELECT
    e.employee_id,
    CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
    e.salary AS employee_salary,
    CONCAT(m.first_name, ' ', m.last_name) AS manager_name,
    m.salary AS manager_salary
FROM employees e
LEFT JOIN employees m
    ON e.manager_id = m.employee_id
ORDER BY e.employee_id;
```

**Expected columns:** employee_id, employee_name, employee_salary, manager_name, manager_salary

**Output:**

| employee_id | employee_name | employee_salary | manager_name | manager_salary |
|---|---|---|---|---|
| 101 | John Smith | 75000.00 | NULL | NULL |
| 102 | Mary Jones | 68000.00 | John Smith | 75000.00 |
| 103 | David Brown | 72000.00 | John Smith | 75000.00 |
| 104 | Linda Taylor | 80000.00 | John Smith | 75000.00 |
| 105 | James Wilson | 55000.00 | Linda Taylor | 80000.00 |
| 106 | Susan Thomas | 60000.00 | Linda Taylor | 80000.00 |
| 107 | Robert Martin | 52000.00 | Linda Taylor | 80000.00 |
| 108 | Karen Lee | 70000.00 | John Smith | 75000.00 |
| 109 | Michael Clark | 65000.00 | John Smith | 75000.00 |
| 110 | Nancy Hall | 85000.00 | John Smith | 75000.00 |
| 111 | Daniel Young | 50000.00 | Linda Taylor | 80000.00 |
| 112 | Emma King | 62000.00 | John Smith | 75000.00 |

### Exercise 7: Complex Aggregation

Find the total revenue by product category. Display category, count of distinct orders, total units sold, and total revenue. Order by total revenue in descending order.

**Hint:** Join products and order_details, calculate revenue as quantity × unit_price.

```sql-- Write your query here

SELECT

    p.category,

    COUNT(DISTINCT od.order_id) AS order_count,

    SUM(od.quantity) AS total_units_sold,

    SUM(od.quantity * od.unit_price) AS total_revenue

FROM products p

JOIN order_details od

    ON p.product_id = od.product_id

GROUP BY p.category

ORDER BY total_revenue DESC;
```

**Expected columns:** category, order_count, total_units_sold, total_revenue

**Output:**

| category | order_count | total_units_sold | total_revenue |
|---|---|---|---|
| Electronics | 7 | 7 | 2640.00 |
| Accessories | 6 | 8 | 8 0.00 |

### Exercise 8: JOINs with WHERE and HAVING

Find customers who have placed more than 1 completed order and their total spending. Display customer ID, customer name, count of completed orders, and total spent. Order by total spent in descending order.

**Hint:** Filter for completed orders using WHERE, then use HAVING to filter groups with more than 1 order.

```sql

-- Write your query here

SELECT

    c.customer_id,

    c.customer_name,

    COUNT(o.order_id) AS completed_orders,

    SUM(o.total_amount) AS total_spent
```

```
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.status = 'Completed'
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) > 1
ORDER BY total_spent DESC;
```

**Expected columns:** customer_id, customer_name, completed_orders, total_spent

**Output:**

| customer_id | customer_name | completed_orders | total_spent |
|---|---|---|---|
| 2 | Alice | 2 | 450.00 |
| 1 | John | 2 | 430.00 |

### Exercise 9: Finding Unmatched Records

Find all products that have never been ordered. Display product ID, product name, category, and unit price. Order by product ID.

**Hint:** Use LEFT JOIN and check for NULL values in the joined table.

```sql
-- Write your query here
SELECT
    p.product_id,
    p.product_name,
    p.category,
    p.unit_price
FROM products p
LEFT JOIN order_details od
    ON p.product_id = od.product_id
WHERE od.product_id IS NULL
ORDER BY p.product_id;
```

**Expected columns:** product_id, product_name, category, unit_price

**Output:**

| product_id | product_name | category | unit_price |
|---|---|---|---|
| 9991 | Wireless Charger | Electronics | 45.00 |
| 9992 | Laptop Stand | Accessories | 30.00 |
| 9993 | USB Hub | Accessories | 25.00 |

**### Exercise 10: Advanced Multi-Table Analysis**

For each country, show the total number of orders, total revenue, and average order value. Only include countries that have orders. Display country, total orders, customer count, total revenue, and average order value. Order by total revenue in descending order.

****Hint:**** Join customers and orders, group by country, and use HAVING to exclude countries with no orders.

```**sql**

**-- Write your query here**

SELECT

  c.country,

  COUNT(o.order_id) AS total_orders,

  COUNT(DISTINCT c.customer_id) AS customer_count,

  SUM(o.total_amount) AS total_revenue,

  AVG(o.total_amount) AS avg_order_value

FROM customers c

JOIN orders o

  ON c.customer_id = o.customer_id

GROUP BY c.country

HAVING COUNT(o.order_id) > 0

ORDER BY total_revenue DESC;
```

****Expected columns:**** country, total_orders, customer_count, total_revenue, avg_order_value

**Output:**

| country | total_orders | customer_count | total_revenue | avg_order_value |
|---|---|---|---|---|
| USA | 10 | 5 | 2275.00 | 227.500000 |
| Canada | 1 | 1 | 900.00 | 900.000000 |
| UK | 3 | 2 | 650.00 | 216.666667 |

---

### Exercise 11: Subquery with GROUP BY

Find departments where the average salary is above the company average. Display department name, department average salary, and company average salary. Order by department average salary in descending order.

**Hint:** Use a subquery in the HAVING clause to compare against the overall average.

```sql
-- Write your query here
SELECT
    d.dept_name,
    AVG(e.salary) AS dept_avg_salary,
    (SELECT AVG(salary) FROM employees) AS company_avg_salary
FROM departments d
JOIN employees e
    ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > (
    SELECT AVG(salary)
    FROM employees
)
ORDER BY dept_avg_salary DESC;
```

**Expected columns:** dept_name, dept_avg_salary, company_avg_salary

**Output:**

| dept_name | dept_avg_salary | company_avg_salary |
|---|---|---|
| HR | 82500.000000 | 66166.666667 |
| IT | 70666.666667 | 66166.666667 |
| Finance | 66666.666667 | 66166.666667 |

### Exercise 12: Date-Based Grouping

Show monthly order statistics for 2024. Display month (YYYY-MM format), order count, monthly revenue, and average order value. Order by month.

**Hint:** Use date functions to extract year-month. Functions vary by database:

- SQLite: `strftime('%Y-%m', order_date)`

- MySQL: `DATE_FORMAT(order_date, '%Y-%m')`

- PostgreSQL: `TO_CHAR(order_date, 'YYYY-MM')

```sql
-- Write your query here
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS month,
    COUNT(order_id) AS order_count,
    SUM(total_amount) AS monthly_revenue,
    AVG(total_amount) AS avg_order_value
FROM orders
WHERE YEAR(order_date) = 2024
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

**Expected columns:** month, order_count, monthly_revenue, avg_order_value

**Output:**

| | month | order_count | monthly_revenue | avg_order_value |
|---|---|---|---|---|
| ▶ | 2024-01 | 3 | 1245.00 | 415.000000 |
| | 2024-02 | 3 | 545.00 | 181.666667 |
| | 2024-03 | 5 | 1645.00 | 329.000000 |
| | 2024-04 | 3 | 390.00 | 130.000000 |

### Exercise 13: Complex Filtering

Find the top 3 products by revenue. Display product ID, product name, category, total quantity sold, total revenue, and times ordered. Order by total revenue in descending order and limit to 3 results.

**Hint:** Calculate revenue as quantity × unit_price, use ORDER BY and LIMIT.

```sql
-- Write your query here
SELECT
    p.product_id,
    p.product_name,
    p.category,
    SUM(od.quantity) AS total_quantity,
    SUM(od.quantity * od.unit_price) AS total_revenue,
    COUNT(DISTINCT od.order_id) AS times_ordered
FROM products p
JOIN order_details od
    ON p.product_id = od.product_id
GROUP BY
    p.product_id,
    p.product_name,
    p.category
ORDER BY total_revenue DESC
LIMIT 3;
```

**Expected columns:** product_id, product_name, category, total_quantity, total_revenue, times_ordered

**Output:**

| product_id | product_name | category | total_quantity | total_revenue | times_ordered |
|---|---|---|---|---|---|
| 301 | Laptop | Electronics | 2 | 1800.00 | 2 |
| 304 | Monitor | Electronics | 3 | 600.00 | 3 |
| 305 | Headphones | Electronics | 2 | 240.00 | 2 |

### Exercise 14: CASE with GROUP BY

Categorize employees by salary range and count employees in each range by department. Salary ranges:

- Entry Level: Less than 60000

- Mid Level: 60000 to 74999

- Senior Level: 75000 and above

Display department name, salary range, employee count, and average salary. Order by department name and then average salary.

**Hint:** Use CASE statement to create salary ranges, include it in both SELECT and GROUP BY.

```sql
-- Write your query here
SELECT
  d.dept_name,
  CASE
    WHEN e.salary < 60000 THEN 'Entry Level'
    WHEN e.salary BETWEEN 60000 AND 74999 THEN 'Mid Level'
    ELSE 'Senior Level'
  END AS salary_range,
  COUNT(e.employee_id) AS employee_count,
  AVG(e.salary) AS avg_salary
FROM departments d
JOIN employees e
  ON d.dept_id = e.dept_id
GROUP BY
  d.dept_name,
  CASE
    WHEN e.salary < 60000 THEN 'Entry Level'
    WHEN e.salary BETWEEN 60000 AND 74999 THEN 'Mid Level'
    ELSE 'Senior Level'
  END
ORDER BY
  d.dept_name,
  avg_salary;
```

**Expected columns:** dept_name, salary_range, employee_count, avg_salary

**Output:**

| dept_name | salary_range | employee_count | avg_salary |
|---|---|---|---|
| Finance | Mid Level | 3 | 66666.666667 |
| HR | Senior Level | 2 | 82500.000000 |
| IT | Mid Level | 2 | 68500.000000 |
| IT | Senior Level | 1 | 75000.000000 |
| Marketing | Entry Level | 2 | 51000.000000 |
| Sales | Entry Level | 1 | 55000.000000 |
| Sales | Mid Level | 1 | 60000.000000 |

Note :Use Ai for logic

**### Exercise 15: Comprehensive Report**

Create a comprehensive customer report for customers with completed orders. Display: customer ID, customer name, country, total orders, total spent, average order value, last order date, and first order date. Order by total spent in descending order.

****Hint:**** Join customers and orders, filter for completed status, use multiple aggregate functions.

```sql
-- Write your query here
SELECT
    c.customer_id,
    c.customer_name,
    c.country,
    COUNT(o.order_id) AS total_orders,
    SUM(o.total_amount) AS total_spent,
    AVG(o.total_amount) AS avg_order_value,
    MAX(o.order_date) AS last_order_date,
    MIN(o.order_date) AS first_order_date
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.status = 'Completed'
GROUP BY
    c.customer_id,
    c.customer_name,
    c.country
ORDER BY total_spent DESC;
```

**Expected columns:** customer_id, customer_name, country, total_orders, total_spent, avg_order_value, last_order_date, first_order_date

**Output:**

| customer_id | customer_name | country | total_orders | total_spent | avg_order_value | last_order_date | first_order_date |
|---|---|---|---|---|---|---|---|
| 2 | Alice | UK | 2 | 450.00 | 225.000000 | 2024-03-20 | 2024-03-05 |
| 1 | John | USA | 2 | 430.00 | 215.000000 | 2024-02-15 | 2024-01-10 |

## Challenge Exercises (Bonus)

These exercises are optional and more advanced. Attempt them if you finish the main exercises early.

### Challenge 1: Employee Hierarchy Analysis

Show each employee's level in the organization hierarchy and their path from the top. This requires a recursive query.

**Hint:** Use a recursive CTE (Common Table Expression) with WITH RECURSIVE.

```sql
-- Write your query here
WITH RECURSIVE emp_hierarchy AS (
  -- Step 1: Anchor query (top-level employees)
  SELECT
    employee_id,
    CONCAT(first_name, ' ', last_name) AS employee_name,
    manager_id,
    1 AS level,
    CONCAT(first_name, ' ', last_name) AS path
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  -- Step 2: Recursive query (employees reporting to managers)
  SELECT
    e.employee_id,
    CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
    e.manager_id,
    eh.level + 1 AS level,
    CONCAT(eh.path, ' → ', e.first_name, ' ', e.last_name) AS path
  FROM employees e
```

```
    JOIN emp_hierarchy eh
        ON e.manager_id = eh.employee_id
)
SELECT
    employee_id,
    employee_name,
    manager_id,
    level,
    path
FROM emp_hierarchy
ORDER BY path;
```

**Expected columns:** employee_id, employee_name, manager_id, level, path

**Output:**

| employee_id | employee_name | manager_id | level | path |
|---|---|---|---|---|
| 101 | John Smith | NULL | 1 | John Smith |
| 103 | David Brown | 101 | 2 | John Smith → David Brown |
| 112 | Emma King | 101 | 2 | John Smith → Emma King |
| 108 | Karen Lee | 101 | 2 | John Smith → Karen Lee |
| 104 | Linda Taylor | 101 | 2 | John Smith → Linda Taylor |
| 111 | Daniel Young | 104 | 3 | John Smith → Linda Taylor → Daniel Young |
| 105 | James Wilson | 104 | 3 | John Smith → Linda Taylor → James Wilson |
| 107 | Robert Martin | 104 | 3 | John Smith → Linda Taylor → Robert Martin |
| 106 | Susan Thomas | 104 | 3 | John Smith → Linda Taylor → Susan Thomas |
| 102 | Mary Jones | 101 | 2 | John Smith → Mary Jones |
| 109 | Michael Clark | 101 | 2 | John Smith → Michael Clark |
| 110 | Nancy Hall | 101 | 2 | John Smith → Nancy Hall |

Note:Use AI for this challenge

**### Challenge 2: Product Affinity Analysis**

Find products that are frequently bought together (appear in the same order at least twice). Display the two product names and how many times they were bought together.

**Hint:** Self-join order_details on order_id, ensure product_id1 < product_id2 to avoid duplicates.

```sql
-- Write your query here
SELECT
   p1.product_name AS product_1,
   p2.product_name AS product_2,
   COUNT(*) AS times_bought_together
FROM order_details od1
JOIN order_details od2
   ON od1.order_id = od2.order_id
  AND od1.product_id < od2.product_id
JOIN products p1
   ON od1.product_id = p1.product_id
JOIN products p2
   ON od2.product_id = p2.product_id
GROUP BY
   p1.product_name,
   p2.product_name
HAVING COUNT(*) >= 2
ORDER BY times_bought_together DESC;
```

**Expected columns:** product_1, product_2, times_bought_together

**Output:**

| | product_1 | product_2 | times_bought_together |
|---|---|---|---|
| ▶ | Keyboard | Monitor | 2 |

Note: Use AI for this

## Tips for Success

1. **Read the requirements carefully** - Make sure you understand what data is requested
2. **Start with the FROM clause** - Identify which tables you need
3. **Add JOINs one at a time** - Build your query incrementally
4. **Test your query** - Run it to see if you get the expected columns
5. **Check your results** - Do the numbers make sense?

6. **Use table aliases** - Makes queries more readable (e.g., `FROM employees e`)

7. **Format your SQL** - Use proper indentation and line breaks

8. **Comment your code** - Add comments to explain complex logic

## Common Mistakes to Avoid

- Forgetting to include columns in GROUP BY that appear in SELECT
- Using WHERE instead of HAVING for filtering aggregated results
- Incorrect JOIN conditions leading to Cartesian products
- Not using LEFT JOIN when you need to include unmatched records
- Forgetting to order results when specified

## Submission Guidelines

1. Save your queries in a file named `firstname_lastname_sql_lab.sql`
2. Include comments with your name and the exercise number
3. Format your SQL code properly with indentation
4. Test all queries before submission
5. Submit your file to your trainer

## Evaluation Criteria

Your work will be evaluated on:

- **Correctness (60%)** - Does the query return the expected results?
- **Query Structure (20%)** - Proper use of JOINs, GROUP BY, and HAVING?
- **Efficiency (10%)** - Is the query reasonably optimized?
- **Code Quality (10%)** - Is the SQL clean, readable, and well-formatted?

Good luck with your exercises!